

SOEN 6011 — Delivery 2

Function F7: x^y

Yichen Huang
Student ID: 40167688

Concordia University

July 28, 2026

- ➊ Problem 5: From-Scratch Implementation and GUI
- ➋ Problem 6: GitHub Repository and Documentation
- ➌ Problem 7: Updated Requirements

Problem 5 — From D1 to D2

Delivery 1

- Textual user interface
- Integer powers by exponentiation by squaring
- Non-integer powers using library logarithm and exponential functions
- Validation through built-in numerical utilities

Delivery 2

- Tkinter graphical user interface
- Integer powers still use exponentiation by squaring
- Natural logarithm implemented from scratch
- Exponential function implemented from scratch
- Custom validation and exception handling

Main Modification

The D1 hybrid algorithm was preserved, but all mathematical operations required for non-integer exponents were reimplemented without using Python power, logarithm, or exponential functions.

Problem 5 — From-Scratch Constraints

Permitted Operations

- Input and output operations
- Basic arithmetic and comparisons
- Explicit while-loop iteration
- Exception handling
- Tkinter graphical-interface operations
- Input parsing and output formatting outside the numerical core

Excluded from the Numerical Core

- `pow()`, `math.pow()`, and `**`
- `math.log()`, `math.exp()`, and `math.isfinite()`
- `range()`, `int()`, and `round()`
- `abs()`, `min()`, and `max()`
- External numerical libraries

Compliance Boundary

The numerical calculation layer uses custom arithmetic, comparisons, and explicit iteration. Input conversion, result formatting, exception handling, and Tkinter remain separate because they support interaction rather than replace the required numerical algorithms.

Compliance Result

No prohibited built-in or library function is used for power evaluation, logarithm, exponential evaluation, finite-value checking, integer detection, or numerical iteration.

Problem 5 — Program Architecture

GUI and Input Layer

- `PowerCalculatorApp`
- Reads base x and exponent y
- Handles Calculate, Clear, and Exit
- Displays results and status messages
- Uses `parse_number()`

Validation and Exceptions

- `is_finite_number()`
- `is_integer_value()`
- Domain and range checks
- Custom exception hierarchy

Calculation Layer

`calculate_power(base,
exponent)`



Integer-valued exponent?

Yes

No



`power_by_
squaring()`

`natural_log()
+ exponential()`

Numeric Protection

`_checked_multiply()` detects overflow and severe underflow.

Problem 5 — Integer Exponent Implementation

Exponentiation by Squaring

- Used when y is integer-valued
- Requires $O(\log |y|)$ iterations
- Uses arithmetic and loop control only
- Does not use `pow()` or `**`

Supported Cases

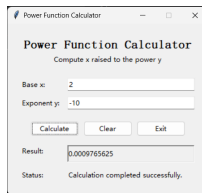
- $y = 0 \Rightarrow x^y = 1$
- $y > 0$: direct integer power
- $y < 0$: invert the base, then use $|y|$
- Negative bases are supported

Negative Exponent

For $y < 0$, the algorithm first replaces the base with its reciprocal, then applies exponentiation by squaring using $|y|$.

Core Logic

```
result = 1
factor = base
remaining = exponent
if remaining < 0:
    factor = 1 / factor
    remaining = -remaining
while remaining > 0:
    multiply when odd
    halve remaining
    square factor
```



Verified Behavior

$$2^{-10} = 0.0009765625$$

- Correct negative result
- Base inverted first
- No false overflow

Problem 5 — Tkinter Graphical User Interface

Interface Components

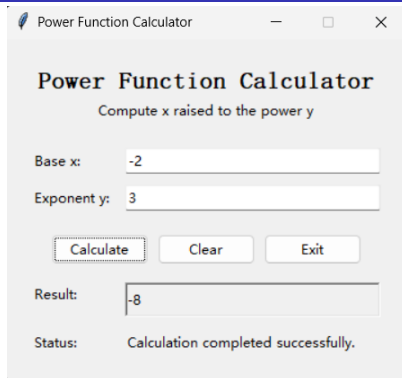
- Labelled fields for base x and exponent y
- Calculate, Clear, and Exit buttons
- Separate result and status areas
- Enter key triggers calculation

Button Behaviour

- **Calculate:** validates and computes
- **Clear:** resets all fields
- **Exit:** closes the application

Usability Goal

The interface remains open after invalid input so the user can correct the values and try again.



Successful GUI calculation: $(-2)^3 = -8$

Implementation

The class `PowerCalculatorApp` manages the widgets, event handling, result formatting, and status messages.

Problem 5 — Exception and Error Handling

Custom Exceptions

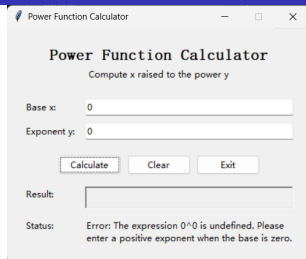
- `InvalidInputError`
- `UnsupportedDomainError`
- `NumericRangeError`
- `ConvergenceError`

GUI Recovery

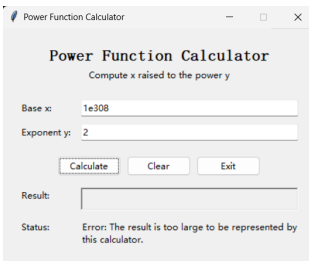
- Catches expected calculator errors
- Clears the result field
- Displays a plain-language message
- Keeps the application open

User Feedback

Messages explain the error and how the input should be corrected.



Domain validation: 0^0 is rejected with guidance.



Numeric-range validation: overflow is detected and reported.

Problem 5 — Verification Results

Base x	Exponent y	Observed Result	Status
2	-10	Result: 0.0009765625	Passed
-2	-3	Result: -0.125	Passed
2	0.3	Result: 1.23114441334492	Passed
10	-400	Severe underflow detected: result too small	Passed
10	400	Overflow detected: result too large	Passed
0	0	Rejected: 0^0 is undefined	Passed
-2	0.5	Rejected: integer exponent required	Passed
abc	2	Rejected: numeric base required	Passed

Independent Accuracy Check

For $2^{0.3}$, the calculator returned 1.23114441334492, compared with the independently verified expected value 1.2311444133449163. The relative error is below 10^{-9} .

Verification Summary

The corrected negative-exponent path produced the expected results. Very small negative-exponent results were classified as underflow, large positive-exponent results were classified as overflow, and the GUI remained open after all expected errors.

Problem 6 — GitHub Repository and Documentation

Public Repository

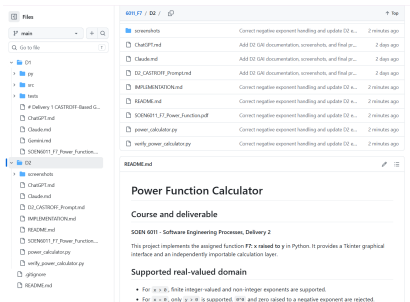
- Hosted publicly on GitHub
- D1 and D2 preserved separately
- Source code, verification, screenshots, documentation, and presentation included
- Root README links directly to both deliverables

Repository:

github.com/prprtracy/6011_F7

Documentation

- Root repository overview
- Detailed D2 README
- Implementation notes
- Verification instructions
- Screenshot and GAI evidence



Repository and Commit Evidence

The repository contains the final D2 code, verification script, documentation, screenshots, GAI evidence, and presentation. The complete commit history was reviewed, and the latest commit records the corrected negative-exponent handling and updated evidence.

Problem 7 — Updated Functional and Input Requirements

ID	Updated Requirement	Change
FR-001	The system shall compute x^y within the supported real-valued domain using from-scratch numerical algorithms.	Modified
IR-001	The system shall accept the base x and exponent y through separately labelled input fields.	Modified
IR-002	The system shall accept only finite numeric values for x and y .	Retained
IR-003	The system shall determine whether y is integer-valued without using prohibited numerical-library functions.	Modified
IR-004	The system shall allow the user to correct invalid input and perform another calculation without restarting the application.	Modified
CR-001	The numerical calculation layer shall not use prohibited built-in or library functions for power evaluation, logarithm, exponential evaluation, finite-value checking, integer detection, or numerical iteration.	Modified
CR-002	The system shall run as a standalone Python program without dependence on any specific integrated development environment.	New

D1-to-D2 Modification

The supported mathematical domain remains consistent with D1. The principal changes are the Tkinter input mechanism, custom numerical validation, IDE-independent execution, and the from-scratch implementation of the subordinate logarithm and exponential functions.

Problem 7 — Updated Output, Error, and Usability Requirements

ID	Updated Requirement	Change
OR-001	The system shall display the computed result in a clearly labelled result area.	Modified
ER-001	The system shall reject $x = 0$ when $y \leq 0$ and display a helpful error message.	Retained
ER-002	The system shall reject $x < 0$ when y is not integer-valued and explain the supported real-valued domain.	Retained
ER-003	The system shall detect overflow, severe underflow, and convergence failure and display a helpful error message.	Modified
ER-004	The system shall recover from invalid input without closing the GUI.	Modified
ER-005	The system shall handle expected input, domain, numeric-range, and convergence errors using specific exception classes without hiding unexpected programming errors.	New
UR-001	The system shall use plain-language labels, status messages, and error messages that explain how the user can correct the input.	Modified
UR-002	The system shall provide a Tkinter graphical user interface for input and output.	Modified
UR-003	The system shall provide Calculate, Clear, and Exit controls and support calculation through the Enter key.	Modified
AR-001	For predefined independently verified test cases, the computed result shall have an absolute error of at most 10^{-9} when the expected result is zero, and a relative error of at most 10^{-9} otherwise.	Retained

Traceability

Each requirement is linked to a GUI element, validation rule, calculation function, exception class, or verification case implemented in Problem 5.

GAI Use in Delivery 2

Part	Prompt Type	GAI Support	Student Work and Verification
P5	Planning, alternatives, and technical review	ChatGPT and Claude suggested possible D1-to-D2 structures, candidate Tkinter interface ideas, exception categories, and iterative approaches for the custom numerical functions.	Selected the final architecture; implemented and revised the production code; integrated the GUI and calculation layers; corrected the negative-exponent, overflow, underflow, and recovery behavior; ran the application; and verified the results.
P6	Repository organization and documentation	Suggested possible repository organization, README topics, documentation structure, screenshot naming, and examples of meaningful commit wording.	Created and organized the actual D2 files; wrote and revised the documentation; captured repository evidence; reviewed the complete commit history; wrote the final commit messages; and verified the public repository.
P7	Requirements drafting and comparison	Suggested candidate ISO/IEC/IEEE 29148-style requirement wording and possible retained, modified, and new classifications.	Compared each candidate requirement with the completed implementation; preserved appropriate D1 identifiers; corrected inaccurate wording; added missing requirements; and finalized the updated list.

Use and Non-Use of GAI

GAI was used for brainstorming, alternatives, wording, comparison, and technical review. The student made the final design decisions, wrote and revised the production implementation, executed and debugged the application, produced the repository artifacts, and finalized all requirements and presentation claims.

CASTROFF Prompt and Student Responsibility

CASTROFF-Based Prompt Structure

Context: SOEN 6011 Delivery 2 for function x^y , extending the completed D1 hybrid implementation.

Audience: The professor, teaching assistant, software-engineering students, and the D1 engineering-student persona.

Scope: Problems 5, 6, and 7 only.

Task: Propose, analyze, and refine candidate approaches for the from-scratch numerical methods, Tkinter GUI, repository documentation, verification, and updated requirements, subject to student implementation and final review.

Restrictions: Do not use prohibited built-in or library functions in the numerical calculation layer. Do not claim that code, screenshots, commits, or repository changes were verified unless actual evidence exists.

Output: Candidate implementation guidance, documentation structure, verification ideas, and requirement revisions for student evaluation and integration.

Final Review: Compare every claim with the project description, source code, verification results, screenshots, README, and GitHub history.

Student Responsibility

GAI outputs were used only as non-authoritative drafts and technical suggestions. The student selected the final design, implemented and revised the production code, integrated the Tkinter GUI, corrected defects, executed and tested the program, created the repository artifacts, finalized the requirements, and verified every presentation claim against actual evidence.

Repository Evidence: The original CASTROFF prompt and GAI outputs are stored as D2/D2_CASTROFF_Prompt.md,

References



Concordia University,
SOEN 6011 Project Description,
Summer 2026.



ISO/IEC/IEEE,
ISO/IEC/IEEE 29148:2018 Systems and Software Engineering — Life Cycle Processes — Requirements Engineering,
2018.



H. Tavakoli,
“The CASTROFF Framework: A Professional Standard for Prompt Engineering,”
in *Prompt Engineering for Everyone*, Apress, 2026.



Python Software Foundation,
Python 3 Documentation: tkinter — Python Interface to Tcl/Tk.



TkDocs,
Tkinter Tutorial.



OpenStax,
Precalculus 2e,
sections on exponential and logarithmic functions.



Y. Huang,
SOEN 6011 F7 GitHub Repository,
https://github.com/prprtracy/6011_F7.git.

Thank You

Questions are welcome.

Yichen Huang
SOEN 6011 — Delivery 2
Function F7: x^y